

Guía de ejercicios 2 - Estado de los componentes y eventos



¡Hola! Te damos la bienvenida a esta nueva guía de ejercicios.

¿En qué consiste esta guía?

En la siguiente guía podrás trabajar los siguientes aprendizajes:

- Imprimir variables del estado declaradas con `useState` en los componentes.
- Modificar variables del estado a partir de interacciones con el usuario.
- Crear formularios en React que contengan múltiples campos.
- Gestionar la información de formularios en React con el hook `useState()`.

En este material de estudio, aprenderemos a manejar estados sobre componentes en React. Para ello, comprenderemos inicialmente qué son los **hooks** y cuáles son sus ventajas y aportes en aplicaciones que necesiten mostrar cierta información a partir de estados definidos; entendiendo los estados como la forma de almacenar información dentro de un componente y que este puede cambiar según acciones o eventos que sean ejecutados por los usuarios.

¡Vamos con todo!



Tabla de contenidos

¿Qué son los estados en React?	3
Setup del proyecto contador	4
Incorporando eventos	6
Incorporando el estado	8
¿Qué es el estado?	8
Agreguemos el estado a nuestro componente	8
El estado de los componentes y eventos en React	10
Validando un formulario con un input	10
Setup del proyecto	10
Detectando cambios en un input	13
Validando el formulario	16
Validando un formulario con múltiples inputs	22
Setup del proyecto	22
Gestionando los estados de nuestros inputs del formulario	25
Ejercicio propuesto	32
Para profundizar	32
Preguntas de cierre	32



¡Comencemos!

¿Qué son los estados en React?

Los estados en React nos permiten almacenar, utilizar y mostrar información actualizada en nuestros componentes. Su manejo facilita la construcción de componentes que sean interactivos y de este modo, mostrar valores o datos a partir de eventos y acciones que realicen los usuarios en el sitio web.

Pensemos en el estado como una memoria a corto plazo que maneja cada componente, el estado permitirá que los componentes mantengan esta información, la usen y la actualicen a medida que ocurren cambios en la aplicación.

Veamos un ejemplo del uso de estado en inicio de sesión:



Imagen 1. Estado en inicio de sesión.

Fuente: Desafío Latam.

Tenemos un componente **formulario** con dos inputs (username y password) y un botón, el estado se inicializa con los valores vacíos para username y password.

Al momento de ingresar los datos en los inputs, el estado cambiará y almacenará sus nuevos valores, ya que al hacer clic, el estado permitirá entregar un mensaje al usuario validando las credenciales.

Veamos otro ejemplo, pero esta vez simulando un carrito de compras.



Imagen 2. Estado en carrito de compras.

Fuente: Desafío Latam.

El estado comienza con el carrito vacío, luego se agrega el producto A y el estado almacena este valor, luego se elimina el producto A y se agrega el producto el B y C; esto quiere decir que mientras los datos permanezcan en el estado, se pueden realizar operaciones como eliminar o actualizar, y finalmente el proceso termina con alguna operación como comprar o vaciar el carrito.

Para aprender sobre estados realizaremos un proyecto con un botón y un contador que incrementa cada vez que se hace clic. Este contador inicia su valor en 0.

Setup del proyecto contador

Al crear un proyecto usando Vite, este nos genera automáticamente el botón y el contador, sin embargo, este es un buen ejemplo para practicar el uso de estado por lo que lo realizaremos desde cero paso a paso.

- **Paso 1:** Creamos un nuevo proyecto React bajo el nombre `contador-simple`.

```
npm create vite
```

- **Paso 2:** Una vez creado el proyecto, realizamos una limpieza de nuestro archivo `App.jsx`, eliminando las importaciones que no se necesitan, incluyendo la de `useState`. También se borra la línea que está antes del `return`, y todo lo que se encuentra dentro de las etiquetas `<>` `</>`.

Debe quedar de la siguiente manera:

```
/* App.jsx */
function App() {
  return (
    <>

    </>
  );
}

export default App;
```



Se recomienda eliminar el logo de Vite que se encuentra en la carpeta `public`, y también el logo de React ubicado en la carpeta `src/assets`.

Esta limpieza consiste en utilizar los archivos e importaciones que realmente necesitamos para lograr el objetivo de nuestro ejercicio.

- **Paso 3:** En la carpeta `/src` crearemos un nuevo directorio llamado `/components`. Dentro de este, agregaremos un archivo llamado `Contador.jsx`.

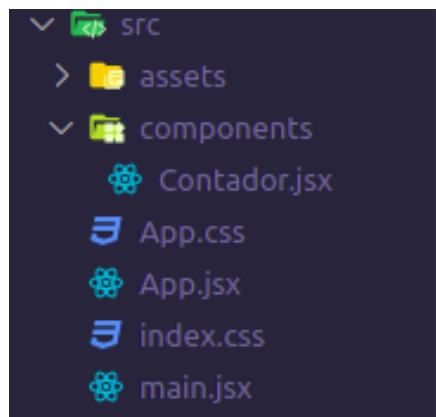


Imagen 3. Componente Contador.jsx
Fuente: Desafío Latam

- **Paso 4:** En el componente contador se mostrará un botón `<button>` con el texto "Cuenta: 0".

```
// components/Contador.jsx

const Contador = () => {
  return (
    <>
      <button> Cuenta: 0</button>
    </>
  )
}

export default Contador
```

- **Paso 5:** Importamos nuestro componente en el `App.jsx` para visualizarlo en el navegador.

```
//App.jsx

import './App.css'
import Contador from './components/Contador';

function App() {
  return (
    <>
      <h1>Contador</h1>
      <Contador />
    </>
  )
}
```

```
);  
}  
  
export default App;
```

- **Paso 6:** Comprobamos que la aplicación funcione bien mostrándose en pantalla el siguiente contenido:

Contador

Cuenta: 0

Imagen 4. Componente Contador cargado
Fuente: Desafío Latam

¡Muy bien! Sigamos con nuestro ejercicio para trabajar con los estados.

Incorporando eventos

En React, al igual que en Javascript, tenemos una variedad de eventos que podemos utilizar, algunos de ellos son:

- ❖ `onClick()`
- ❖ `onSubmit()`
- ❖ `onChange()`
- ❖ Entre otros.

Para nuestro contador utilizaremos `onClick()`. El resto de los eventos los estudiaremos más adelante en esta guía.



En React los eventos debemos definirlos con camelCase y siempre comenzarán con la palabra "on".

- **Paso 7:** En nuestro componente `Contador.jsx` agregaremos en el botón el evento `onClick()` para ejecutar la función `console.log("Hola")`. Veamos cómo debe quedar el código del componente.

```
// components/Contador.jsx

const Contador = () => {
  return(
    <>
      <button onClick={() => (console.log("hola"))}>
        Cuenta: 0
      </button>
    </>
  )
}

export default Contador
```

- **Paso 8:** Comprobamos que al dar click sobre el boton, se imprime en la consola del navegador el texto "hola". Debe mostrarse como en la siguiente imagen:



Imagen 5. Impresión en consola evento onClick
Fuente: Desafío Latam

Incorporando el estado

El siguiente paso de nuestro ejercicio consiste en modificar el contador. En React para guardar valores que puedan cambiar, necesitamos hacer uso del estado.

¿Qué es el estado?

El estado es un objeto interno del componente, donde podemos guardar todos los valores que necesitemos. Para usar estados, ocuparemos una función especial llamada **useState** (Traducido, es literalmente usar estado).

Para utilizar useState necesitamos:

- a. Importar useState al inicio del componente.

```
import { useState } from "react";
```

- b. Definir un nombre de variable que representará el estado que queremos guardar.
- c. Definir un nombre de una función que cambiará el estado.
- d. Un valor inicial para el estado, el cual puede ser un valor de tipo *number*, *string*, *array*, *object*, *boolean*.

```
const [state, setState] = useState(valor_inicial);
```

Agreguemos el estado a nuestro componente

Continuemos con el ejercicio e incorporemos `useState()`.

- **Paso 9:** Importamos `useState()` en el componente `Contador.jsx`, luego definimos una variable llamada `cuenta` que almacenará nuestro estado y el modificador lo llamaremos `setCuenta()`, el estado inicial será 0.

```
// components/Contador.jsx

import {useState} from 'react'

const Contador = () => {
  const [cuenta, setCuenta] = useState(0) // Definimos un estado
  llamado cuenta con valor 0, la función setCuenta podrá cambiar este
  estado.
  return(
```



```
    <>
      <button onClick={() => (console.log("hola"))}> Cuenta: {cuenta}
    </button>
    </>
  )
}

export default Contador
```

Ahora dentro de nuestro componente tenemos una cuenta con valor inicial cero y una función para cambiar el valor que ocuparemos en el próximo paso.

- **Paso 10:** Luego, hacemos uso de la función `setCuenta` para aumentar el valor de la cuenta. Para ello cambiaremos el código del clic para que lo haga.

```
// Contador.jsx
import {useState} from 'react'

const Contador = () => {
  const [cuenta, setCuenta] = useState(0)

  return(
    <>
      <button onClick={() => setCuenta(cuenta + 1)}> Cuenta: {cuenta}
    </button>
    </>
  )
}

export default Contador
```

- **Paso 11:** Observemos en el navegador el comportamiento esperado al darle clic al botón.

Contador

Cuenta: 1

Imagen 6. Resultado `useState`
Fuente: Desafío Latam



Nota: Podrás acceder al repositorio de este ejercicio en el LMS con el nombre Repositorio Ejercicio - Contador de clics en React.



Recuerda que para usar el repositorio debes descargarlo, abrirlo en la terminal y correr el comando `npm install`.



¡Felicitaciones! Es normal que el primer uso de `useState` nos parezca poco familiar, hasta ahora no habíamos ocupado funciones de esta forma y tampoco habíamos trabajado con una función como `useState` que devuelve una función, revisando algunos ejercicios más lo dominaremos.

El estado de los componentes y eventos en React

En este material de estudio, seguiremos trabajando con `useState()` pero en esta ocasión lo utilizaremos para manejar estados en respuesta al cambio, envío de inputs y formularios, de forma de utilizarlo en conjunto con otro tipo de eventos.

Validando un formulario con un input

Uno de los usos más frecuentes es la validación de inputs, para aprender sobre esto crearemos un proyecto nuevo llamado `estados-formulario` donde construiremos un componente que será un formulario con un campo de tipo input y un botón de envío.

Mostraremos un mensaje de error al usuario cuando el input esté vacío.

¡Veamos el siguiente ejercicio!: estados y formularios.

Setup del proyecto

- **Paso 1:** Creamos una nueva aplicación en React llamada `estados-formulario`.

```
npm create vite
```

- **Paso 2:** Limpiamos el archivo `App.jsx` eliminando las importaciones que no son necesarias y eliminando el código que se encuentra dentro del `return`.

Recuerda depurar los archivos que no se están importando o mostrando para que la aplicación funcione correctamente.

- **Paso 3:** En `/src` creamos una carpeta `/components` y agregamos un componente `Formulario.jsx`.
- **Paso 4:** Dentro del archivo `App.jsx` dejamos un código básico para iniciar:

```
// App.jsx
import Formulario from './components/Formulario';

function App() {
  return (
    <>
      <h1>Ejercicio formulario</h1>
      <Formulario/>
    </>
  );
}

export default App;
```

- **Paso 5:** En el componente `Formulario.jsx` retornaremos un formulario con un input y un botón de enviar.

```
// components/Formulario.jsx

const Formulario = () => {
  return (
    <>
      <form>
        <input name="Nombre"/>
        <button type="submit">Enviar</button>
      </form>
    </>
  )
}

export default Formulario
```

- **Paso 6:** Importamos los estilos de bootstrap mediante su [CDN](#). Esto lo agregamos en el archivo `index.html` ubicado en la raíz del proyecto.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
    <title>Vite + React</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.3/dist/css/bootstrap.mi
n.css" rel="stylesheet"
integrity="sha384-rbsA2VBKQhggwzxH7pPCaAqO46MgnOM80zW1RWuH61DGLwZJEdK2Ka
dq2F9CUG65" crossorigin="anonymous">
    </head>
    <body>
      <div id="root"></div>
      <script type="module" src="/src/main.jsx"></script>
      <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.2.3/dist/js/bootstrap.bund
le.min.js"
integrity="sha384-kenU1KFdBIe4zVF0s0G1M5b4hcxpyD9F7jL+jjXkk+Q2h455rYXK/7
HAuoJl+0I4" crossorigin="anonymous"></script>
    </body>
  </html>
```

- **Paso 7:** Agregamos clases de bootstrap al formulario, tanto al input como al botón.

```
// components/Formulario.jsx
const Formulario = () => {
  return (
    <>
      <form>
        <div className="form-group">
          <input className="form-control" name="Nombre"/>
          <button className="btn btn-dark mt-3" type="submit">
Enviar</button>
        </div>
      </form>
    </>
  )
}
```

```
}  
  
export default Formulario
```

- **Paso 8:** Probamos nuestro sitio en el navegador y deberíamos ver el input y el botón creado con las configuraciones de Bootstrap.

Ejercicio formulario



Imagen 7. Aplicación con estilos bootstrap.
Fuente: Desafío Latam

¡Felicitaciones! Hasta este punto has creado una aplicación en React incorporando un formulario y estilos con Bootstrap mediante su CDN.

Detectando cambios en un input

Buscamos validar si el input está vacío y según eso mostrar un mensaje en caso de que lo esté. Para lograrlo, guardaremos el contenido del input como estado con `useState`, donde el estado inicial será un string vacío, ya que el input comenzará vacío. Cuando se modifique el input guardaremos el nuevo valor, y para capturar el evento de que el input fue modificado utilizaremos el evento `onChange()`.

Continuemos el ejercicio anterior con los siguientes pasos:

- **Paso 9:** Agregamos un estado al input, utilizaremos `useState()` donde definiremos:
 - Una variable de estado llamado `nombre`.
 - Un setter llamado `setNombre`.
 - Un valor inicial que será un string vacío.
 - También agregaremos el listener de `onchange` al input.

```
// components/Formulario.jsx  
  
import {useState} from 'react'
```

```
const Formulario = () => {
  const [nombre, setNombre] = useState("")

  return (
    <>
      <form>
        <div className="form-group">
          <input className="form-control" name="Nombre"
onChange={(e) => console.log(e)}/>
          <button className="btn btn-dark mt-3" type="submit">
Enviar</button>
        </div>
      </form>
    </>
  )
}

export default Formulario
```

- **Paso 10:** Si observamos la consola del navegador para verificar el comportamiento del evento `onChange()` y la función `console.log()` que está recibiendo, veremos que si escribimos caracteres en el input se imprimen cada uno de los eventos de esta acción.

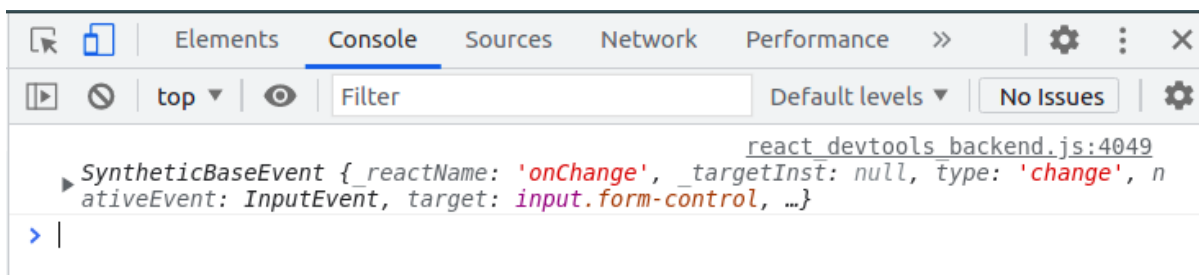


Imagen 8. Evento onChange en consola
Fuente: Desafío Latam

- **Paso 11:** Modifiquemos el `console.log()` definido en el `onChange()` y en vez de pasar el evento, pasaremos `e.target.value`. Esto nos debería mostrar directamente los caracteres que se están agregando al input del formulario.

```
// components/Formulario.jsx
import {useState} from 'react'

const Formulario = () => {
  const [nombre, setNombre] = useState("")
```

```
return (  
  <>  
    <form>  
      <div className="form-group">  
        <input className="form-control" name="Nombre"  
onChange={(e) => console.log(e.target.value)}/>  
        <button className="btn btn-dark mt-3" type="submit">  
Enviar</button>  
      </div>  
    </form>  
  </>  
)  
}  
  
export default Formulario
```

- **Paso 12:** Observemos qué ocurre ahora en la consola del navegador si escribimos en el input la palabra hola.



Imagen 9. Muestra caracteres del input
Fuente: Desafío Latam

- **Paso 13:** Mostremos en pantalla nuestro estado inicial y modifiquemos su valor con el `setNombre`. Para esto, agregamos un `<h3>` con el nombre, y reemplazamos el `console.log` por `setNombre` dentro del evento `onChange`. Nuestro componente `Formulario.jsx` quedará de la siguiente manera:

```
// components/Formulario.jsx  
import {useState} from 'react'  
  
const Formulario = () => {
```

```
const [nombre, setNombre] = useState("")
return (
  <>
    <form>
      <h3>{nombre}</h3>
      <div className="form-group">
        <input className="form-control" name="Nombre"
onChange={(e) => setNombre(e.target.value)} />
        <button className="btn btn-dark mt-3" type="submit">
Enviar</button>
      </div>
    </form>
  </>
)
}
export default Formulario
```

- **Paso 14:** Si observamos los cambios en el navegador, veremos que al escribir en el input se muestra antes del nombre lo que el usuario ingresa. Veamos la siguiente imagen:

Ejercicio formulario

Mi nombre

Imagen 10. Formulario y sitio dinámico
Fuente: Desafío Latam

El texto del input lo estamos mostrando temporalmente. Ahora tenemos que validar el formulario al momento de que el usuario intente enviarlo, si cumple las reglas, o sea en este caso que el input no esté vacío, lo enviaremos, en caso contrario mostraremos un mensaje de error. Para esto utilizaremos tanto la etiqueta form con `onSubmit()`.

Validando el formulario

- **Paso 15:** En el componente `Formulario.jsx`, agregamos a la etiqueta `<form>` de apertura el evento `onSubmit()`, esta recibirá una función que llamaremos `validarInput`. La función la definiremos antes del `return()` y debajo de nuestro state, en la cual vamos a mostrar un `alert()` de JavaScript para verificar que al

oprimir el botón se dispare. Veamos el código a continuación y verifiquemos el funcionamiento en el navegador.

```
// components/Formulario.jsx
import {useState} from 'react'

const Formulario = () => {
  const [nombre, setNombre] = useState("")

  const validarInput = () => {
    alert('evento form')
  }

  return (
    <>
      <form onSubmit={validarInput}>
        <h3>{nombre}</h3>
        <div className="form-group">
          <input className="form-control" name="Nombre"
onChange={(e) => setNombre(e.target.value)}/>
          <button className="btn btn-dark mt-3" type="submit">
Enviar</button>
        </div>
      </form>
    </>
  )
}

export default Formulario
```

En el navegador debe mostrarse nuestro `alert()` de la siguiente manera:

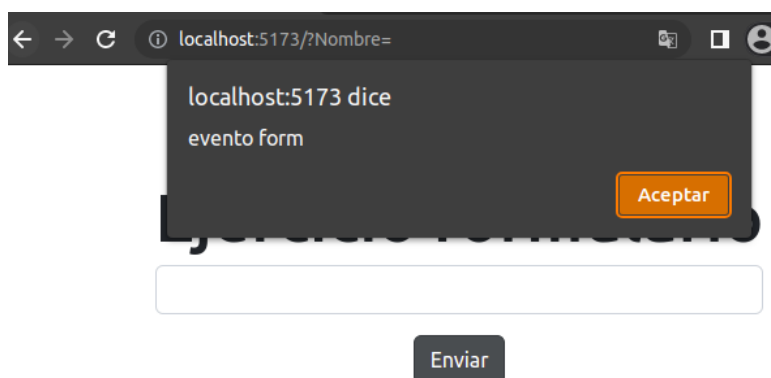


Imagen 11. Evento onSubmit
Fuente: Desafío Latam

¡Excelente! Logramos acceder al evento cuando se oprime el botón de enviar, sigamos con la validación.



Si observas el comportamiento del formulario, verás que el navegador realiza una petición GET y recarga el sitio. Este comportamiento por defecto del botón de tipo `submit` podemos prevenirla con el método `e.preventDefault()`.

- **Paso 16:** Implementemos dentro de la función `validarInput` la lógica de JavaScript correspondiente para que el formulario no pueda ser enviado sin datos ingresados.

```
// components/Formulario.jsx
import {useState} from 'react'

const Formulario = () => {
  const [nombre, setNombre] = useState("")

  const validarInput = (e) => {
    // Prevenimos el comportamiento por defecto
    e.preventDefault()

    // Validación input
    if(nombre === '') {
      alert('Debes agregar tu nombre')
    }
  }

  return (
    <>
      <form onSubmit={validarInput}>
        <h3>{nombre}</h3>
        <div className="form-group">
          <input className="form-control" name="Nombre"
onChange={(e) => setNombre(e.target.value)}/>
          <button className="btn btn-dark mt-3" type="submit">
Enviar</button>
        </div>
      </form>
    </>
  )
}

export default Formulario
```

De esta forma, le estamos diciendo al sistema que cuando el formulario intente ser enviado con el campo nombre vacío, levante un alerta que avise que “Debes agregar tu nombre”.

Quizás hacer esto con un `alert()` no sea lo conveniente, sino mostrar en su defecto un mensaje de error, veamos cómo hacerlo.

- **Paso 17:** Creamos un nuevo estado para mostrar errores, lo llamaremos `[error, setError]`. Este estado mostrará en pantalla un mensaje al usuario si intenta enviar el formulario con el campo input vacío. Veamos el siguiente código y luego analicemos lo que se implementó.

```
// components/Formulario.jsx
import {useState} from 'react'

const Formulario = () => {
  const [nombre, setNombre] = useState("")
  const [error, setError] = useState(false)

  const validarInput = (e) => {
    //Prevenimos el comportamiento por defecto
    e.preventDefault()

    //Validación input
    if(nombre === '') {
      setError(true)

      return
    }
    //Eliminar mensaje de error
    setError(false)
  }

  return (
    <>
      <form onSubmit={validarInput}>
        {error ? <p className="error">Debes ingresar tu nombre</p> :
null}
        <h3>{nombre}</h3>
        <div className="form-group">
          <input className="form-control" name="Nombre"
onChange={(e) => setNombre(e.target.value)}>
          <button className="btn btn-dark mt-3" type="submit">
Enviar</button>
        </div>
      </form>
    </>
  )
}
```

```
        </form>
      </>
    )
  }

  export default Formulario
```

El mensaje de error lo estamos manejando mediante un estado inicial definido como `false`. Una vez que el formulario es enviado con el campo nombre vacío, React modificará el estado a través del `setError(true)`, cambiando de falso a verdadero.

Seguidamente, en la parte inicial de nuestro formulario, estamos validando mediante operador ternario que, en caso de que error sea `true`, e imprima un párrafo con el texto “Debes ingresar tu nombre”, y que en caso contrario no muestre nada con la instrucción `null`.

Si continuamos analizando, en nuestra validación inicial y debajo del `setError(true)` estamos pasando un `return`. Esto lo hacemos dado que en caso de que si existe el error no queremos que se siga ejecutando la lógica siguiente.

Luego, por fuera del `if` le estamos diciendo al sistema que en caso de que el formulario si reciba los datos en el input, entonces elimine el mensaje de error.

Nuestra aplicación se verá de la siguiente manera:



Ejercicio formulario

Mi nombre

Enviar

Imagen 12. Formulario con error true
Fuente: Desafío Latam

Ejercicio formulario

Debes ingresar tu nombre

Enviar

Imagen 13. Formulario con error en false
Fuente: Desafío Latam

- **Paso 18:** Para los estilos de la etiqueta párrafo con el mensaje de error puedes utilizar el siguiente código en nuestro archivo `App.css`.

```
.error {  
  padding: 10px;  
  color: white;  
  background-color: red;  
  text-align: center;  
  font-weight: bold;  
  border-radius: 10px;  
}
```



Nota: podrás acceder al repositorio de este ejercicio en el LMS con el nombre Repositorio Ejercicio - Estados y Formularios.



¡Felicitaciones! En esta ocasión aprendiste a desarrollar un pequeño formulario implementando dos eventos importantes para gestionar el comportamiento por defecto de los mismos. Estos eventos fueron `onChange()` y `onSubmit()`, los cuales son muy utilizados y es por ello que en los materiales de estudio siguientes, continuaremos ejercitando.



Te invitamos a replicar los pasos anteriores sin recurrir a la guía. Puedes modificar la temática del ejercicio e ir probando cosas nuevas, considera que mientras más proyectos y ejercicios realizas, más rápido será tu aprendizaje.

Validando un formulario con múltiples inputs

A continuación seguiremos ejercitando los conceptos adquiridos hasta el momento. Trabajaremos con formularios y veremos cómo manejar el estado de nuestros componentes asignando como valor inicial un objeto con distintos valores por defecto, los cuales en el estado serán los campos de nuestro formulario.

Setup del proyecto

¡Veamos el siguiente ejercicio!: validando múltiples campos de formularios.

- **Paso 1:** Setup del proyecto, creamos una nueva aplicación en React usando Vite, a la cual le daremos el nombre de `formularios-react`.

```
npm create vite
```



Modifica el archivo `App.jsx` eliminando las referencias a los archivos borrados para que la aplicación funcione correctamente.

- **Paso 2:** en `/src` creamos una carpeta `/components` y agregamos un componente `Formulario.jsx`.
- **Paso 3:** dentro del archivo `App.jsx` dejamos un código básico para iniciar:

```
// App.jsx
import './App.css';
import Formulario from './components/Formulario';

function App() {
  return (
    <>
      <Formulario />
    </>
  );
}

export default App;
```

- **Paso 4:** En el componente `Formulario.jsx` retornaremos un formulario que contendrá los campos de nombre, apellido, edad y email. Para cada uno de estos campos, vamos a definir un `label` correspondiente. Veamos cómo quedaría el componente.

```
const Formulario = () => {
  return (
    <>
      <form className="formulario">
        <div className="form-group">
          <label>Nombre</label>
          <input
            type="text"
            name="nombre"
            className="form-control"
          />
        </div>
        <div className="form-group">
          <label>Apellido</label>
          <input
            type="text"
            name="apellido"
            className="form-control"
          />
        </div>
        <div className="form-group">
          <label>Edad</label>
          <input
            type="text"
            name="edad"
            className="form-control"
          />
        </div>
        <div className="form-group">
          <label>Email</label>
          <input
            type="text"
            name="email"
            className="form-control"
          />
        </div>
        <button type="submit" className="btn
btn-primary">Enviar</button>
      </form>
    </>
  )
}
```

```
    </>
  )
}

export default Formulario
```

- **Paso 5:** Importamos los estilos de bootstrap mediante su [CDN](#). Esto lo agregamos en el archivo `index.html` ubicado en la raíz del proyecto `/public`.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
    <title>Vite + React</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.2.3/dist/css/bootstrap.mi
n.css" rel="stylesheet"
integrity="sha384-rbsA2VBKQhggwzxH7pPCaAqO46MgnOM80zW1RWuH61DGLwZJEdK2Ka
dq2F9CUG65" crossorigin="anonymous">
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.jsx"></script>
    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.2.3/dist/js/bootstrap.bund
le.min.js"
integrity="sha384-kenU1KFdBIe4zVF0s0G1M5b4hcxpyD9F7jL+jjXkk+Q2h455rYXK/7
HAuoJl+0I4" crossorigin="anonymous"></script>
  </body>
</html>
```

- **Paso 6:** Probamos nuestro sitio en el navegador y deberíamos ver los inputs y el botón creado con los estilos de Bootstrap.

Nombre

Apellido

Edad

Email

Enviar

Imagen 14. Formulario Bootstrap
Fuente: Desafío Latam



¡Felicitaciones! Hasta este punto has creado una aplicación en React incorporando un formulario y estilos con Bootstrap mediante su CDN.

Gestionando los estados de nuestros inputs del formulario

Para cada campo del formulario asignaremos un estado, esto nos permitirá controlar y almacenar los valores que sean ingresados por los usuarios.

- **Paso 7:** Agregamos el estado `datos` al componente `Formulario.jsx`, el cual recibirá las propiedades definidas en nuestros campos del formulario:

```
// /components/Formulario.jsx
import { useState } from 'react';

const Formulario = () => {
  //Estados del formulario
  const [nombre, setNombre] = useState('');
  const [apellido, setApellido] = useState('');
  const [edad, setEdad] = useState('');
  const [email, setEmail] = useState('');
```

- **Paso 8:** Agregamos a través del evento `onChange()` la función correspondiente que detectará cuando el usuario escriba en los campos del formulario. Este evento va a

recibir un callback que captura a través del `event` los datos que sean ingresados en cada input:

```
import {useState} from 'react'

const Formulario = () => {
  //Estados del formulario
  const [nombre, setNombre] = useState('');
  const [apellido, setApellido] = useState('');
  const [edad, setEdad] = useState('');
  const [email, setEmail] = useState('');

  return (
    <>
      <form className="formulario">
        <div className="form-group">
          <label>Nombre</label>
          <input
            type="text"
            name="nombre"
            className="form-control"
            onChange={(e) => setNombre(e.target.value)}
            value={nombre}
          />
        </div>
        <div className="form-group">
          <label>Apellido</label>
          <input
            type="text"
            name="apellido"
            className="form-control"
            onChange={(e) => setApellido(e.target.value)}
            value={apellido}
          />
        </div>
        <div className="form-group">
          <label>Edad</label>
          <input
            type="text"
            name="edad"
            className="form-control"
            onChange={(e) => setEdad(e.target.value)}
            value={edad}
          />
        </div>
      </form>
    </>
  )
}
```

```
    </div>
    <div className="form-group">
      <label>Email</label>
      <input
        type="email"
        name="email"
        className="form-control"
        onChange={(e) => setEmail(e.target.value)}
        value={email}
      />
    </div>
    <button type="submit" className="btn btn-primary">
      Enviar
    </button>
  </form>
</>
)
}
export default Formulario
```

Si inspeccionamos nuestro componente con *React Developer Tools*, veremos que por cada información ingresada en los campos del formulario, se va agregando a nuestro estado, tal como lo muestra la imagen:

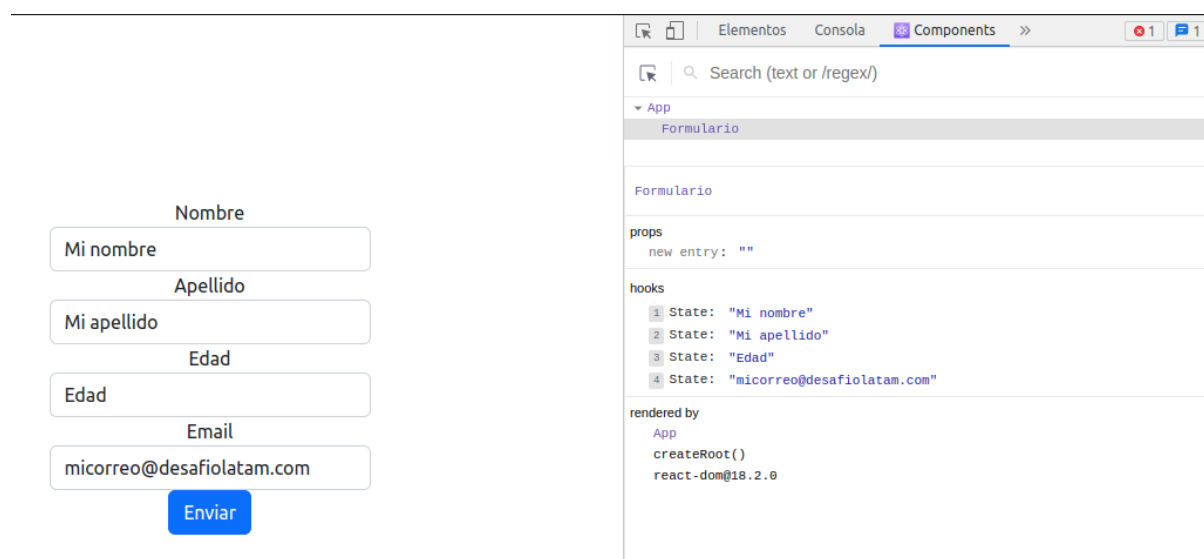


Imagen 15. Estado campo nombre

Fuente: Desafío Latam

Continuemos con nuestro ejercicio, ahora vamos a validar cada input del formulario.

- **Paso 9:** Validamos el evento `onSubmit()` cuando el usuario envíe el formulario. Esta validación la haremos para evitar que los campos estén vacíos. Para ello crearemos una función que se llamará `validarDatos()`, la cual estará incorporada en la etiqueta `<form>` de apertura.



Recuerda prevenir la acción por defecto en la función `validarDatos()` con: `e.preventDefault()`.

El cuerpo de nuestra función será de la siguiente manera:

```
//Función antes de enviar el formulario
const validarDatos = (e) => {
  e.preventDefault()

  //Validación
}
```

- **Paso 10:** Dentro de nuestra función `validarDatos()` haremos a continuación la validación condicional haciendo uso del condicional `if` de JavaScript.

```
//Función antes de enviar el formulario
const validarDatos = (e) => {
  e.preventDefault()

  //Validación
  if (!nombre.trim() || !apellido.trim() || !edad.trim() ||
!email.trim()) {
    alert("Todos los campos son obligatorios");
    return;
  }
}
```

Observa este comportamiento en el navegador y revisa si los campos del formulario se envían vacíos, se dispara la función `alert()` y si está el mensaje definido en su interior.

- **Paso 11:** Como vimos en el ejercicio anterior, podemos gestionar errores a través de estado, por lo que configuremos nuestro componente para implementar errores en la validación. Para ello, debemos crear un nuevo estado `[error, setError]`, el que se activará cuando los campos estén vacíos y se desactive cuando sean completados los campos faltantes.

```
// /components/Formulario.jsx
import { useState } from 'react';

const Formulario = () => {
  //Estados del formulario
  const [nombre, setNombre] = useState('');
  const [apellido, setApellido] = useState('');
  const [edad, setEdad] = useState('');
  const [email, setEmail] = useState('');

  //Estado para los errores
  const [error, setError] = useState(false);

  //Función antes de enviar el formulario
  const validarDatos = (e) => {
    e.preventDefault();

    //Validación;
    if (!nombre.trim() || !apellido.trim() || !edad.trim() ||
!email.trim()) {
      setError(true);
      return;
    }
    setError(false);
  };

  return (
    <>
      <form className="formulario" onSubmit={validarDatos}>
        {error ? <p>Todos los campos son obligatorios</p> : null}
        <div className="form-group">
          <label>Nombre</label>
          <input
            type="text"
            name="nombre"
            className="form-control"
            onChange={(e) => setNombre(e.target.value)}
            value={nombre}
          />
        </div>
        <div className="form-group">
          <label>Apellido</label>
          <input
```

```
        type="text"
        name="apellido"
        className="form-control"
        onChange={(e) => setApellido(e.target.value)}
        value={apellido}
      />
    </div>
    <div className="form-group">
      <label>Edad</label>
      <input
        type="text"
        name="edad"
        className="form-control"
        onChange={(e) => setEdad(e.target.value)}
        value={edad}
      />
    </div>
    <div className="form-group">
      <label>Email</label>
      <input
        type="email"
        name="email"
        className="form-control"
        onChange={(e) => setEmail(e.target.value)}
        value={email}
      />
    </div>
    <button type="submit" className="btn btn-primary">
      Enviar
    </button>
  </form>
  <hr />
  <h1>Datos ingresados</h1>
  {nombre} - {apellido} - {edad} - {email}
</>
);
};

export default Formulario;
```

Imaginemos un caso, ¿qué tendríamos que configurar para que los campos del formulario se vuelvan a mostrar vacíos una vez que el usuario lo envía?

- ¿Te imaginas cómo podría ser esta configuración?

- ¿Identificas con facilidad el código necesario para esto?

Si no logras dar con la solución no temas, estás en proceso de aprendizaje y poco a poco estos casos imaginativos van a ser solucionados con práctica. A continuación sigamos con los pasos.

- **Paso 12:** Después del `setError(false)`, le diremos a nuestro formulario que vuelva al estado inicial, pasando cada una de las funciones setter asociadas a cada campo del formulario a un string vacío, tal como lo definimos desde un inicio. Veamos cómo queda entonces el cuerpo de nuestra función `validarDatos()`.

```
//Función antes de enviar el formulario
const validarDatos = (e) => {
  e.preventDefault();

  //Validación;
  if (!nombre.trim() || !apellido.trim() || !edad.trim() ||
!email.trim()){
    setError(true);
    return;
  }

  // Si el formulario se envía correctamente devolvemos todos nuestros
  estados al inicial y reseteamos el formulario
  setError(false);
  setNombre('');
  setApellido('');
  setEdad('');
  setEmail('');
};
```



Nota: podrás acceder al repositorio de este ejercicio en el LMS con el nombre Repositorio Ejercicio - Validando múltiples campos de formularios.



Recuerda que para usar el repositorio debes descargarlo, abrirlo en la terminal y correr el comando `npm install`.

Ejercicio propuesto



Te invitamos a que sigas agregando validaciones al formulario, estas son algunas que puedes realizar:

- **Confirmación de email:** Crea otro input para confirmar que ambos email sean idénticos, además de validar que ambos tengan el formato correcto.
- **Validación de edad mínima:** Válida que solo usuarios mayores de edad puedan enviar el formulario.
- **Selección de género:** Crea un campo para *seleccionar* el género (Femenino, Masculino, Otro).



Para profundizar

Tomemos unos minutos para autorreflexionar los conocimientos adquiridos hasta el momento respondiendo las siguientes preguntas:

- ¿Cuánto comprendí de los conceptos vistos hasta este punto?
- ¿Cómo puedo relacionar esta información con los conocimientos previos durante mi ciclo formativo?
- ¿Qué aprendí que no quiero olvidar sobre lo visto en este material?

Preguntas de cierre

- ¿Cuáles son los distintos tipos de datos que pueden recibir los estados de nuestros componentes?
- ¿Para qué nos sirve el *spread operator*?
- Si tenemos el atributo `name=nombre`, `name=apellido` en el formulario, ¿cómo se deberá llamar nuestra propiedad en el estado?